Danilo Duarte de Jesus Peixoto

Roteirização de Picking utilizando Algoritmo Genético

São Paulo - Brasil
2022

Danilo Duarte de Jesus Peixoto

Roteirização de Picking utilizando Algoritmo Genético

Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Professor orientador: Alessandro Ranulfo Lima Nery

São Paulo - Brasil
2022

Resumo

A otimização dos processos logísticos dentro de uma empresa da área do varejo é de grande importância, processos claros e otimizados tornará o armazém mais produtivo. Em vista disso, este trabalho propõe o desenvolvimento de um algoritmo genético na linguagem de programação python, para otimizar um processo logístico chamado *picking* (separação de pedidos), comparando os resultados do algoritmo com uma estratégia de roteirização de forma decrescente com base nos endereços de armazenagem dos produtos. Os resultados obtidos mostraram que o algoritmo é eficiente para otimizar a rota de *picking*, porém para grandes entradas o tempo de execução é relativamente alto, portanto se utilizado em um cenário real, será necessário uma análise do tamanho de lote ideal, considerando a taxa de entrada de pedidos do armazém, para que a geração da rota não atrapalhe o desempenho dos *pickers*.

Palavras-chave: Algoritmos genéticos, Roteirização de picking, Gerenciamento de armazém, Logística

Abstract

The optimization of logistical processes within a company in the retail area is of great importance, clear and optimized processes will make the warehouse more productive. In view of this, this work proposes the development of a genetic algorithm in the python programming language, to optimize a logistical process called picking, comparing the results of the algorithm with a routing strategy in descending order based on the storage addresses of the products. The results obtained showed that the algorithm is efficient to optimize the picking route, but for large entries the execution time is relatively high, so if used in a real scenario, an analysis of the ideal lot size will be necessary, considering the rate of entry of orders from the warehouse, so that the generation of the route does not affect the performance of the pickers.

Keywords: Genetic algorithms, Picking routing, Warehouse management, Logistics

Lista de Figuras

Figura 2.1 – Grafos 16
Figura 2.2 – Crossover 19
Figura 2.3 – Fluxograma Algoritmo genético 20
Figura 2.4 – Exemplo do Método de Clark & Wright 24
Figura 3.1 – Grafo Armazém 25
Figura 3.2 – Grafo visualização superior 26
Figura 3.3 – Diagrama tabelas 27
Figura 3.4 – Fluxograma Algoritmo roteirização de picking 31
Figura 3.5 – Médias e ganhos máximo e mínimo 32
Figura 3.6 – Média de tempo por quantidade de produtos 32
Figura 3.7 – Lotes ganho máximo e mínimo 33

Sumário

1 Introdução **11**
1.1 **Contexto** 11
1.2 Problemática 11
1.3 **Objetivos** 12
1.3.1 Objetivo geral 12
1.3.2 Objetivos específicos 12
1.4 Justificativa 12
2 Revisão bibliográfica **13**
2.1 Pesquisa operacional 13
2.2 Logística 14
2.2.1 Processos de logística 14
2.2.1.1 Separação de pedidos 14
2.3 Grafos 15
2.4 Complexidade Computacional 16
2.4.1 Problema do Caixeiro Viajante 17
2.5 Algoritmos Genéticos 17
2.5.1 Funcionamento do algoritmo 18
2.5.2 Aplicações 20
2.6 Trabalhos Relacionados 22
2.6.1 *An algorithm for dynamic order-picking in warehouse operations* . . 22
2.6.2 Proposta de um modelo heurístico para a roteirização de empilhadeiras em um armazém de grande porte 23
3 Desenvolvimento **25**
3.1 Elaboração do modelo de grafo 25
3.2 Elaboração do modelo Matemático 27
3.3 Implementação do algoritmo genético para roteirização 29
3.4 Resultados 32
4 Conclusão e trabalhos futuros **35**
Referências **37**

1 Introdução

1.1 Contexto

Processos claros e otimizados é um dos passos para a excelência operacional, tendo isso consequentemente tornará a operação de um armazém mais produtiva e organizada.

No cenário atual das empresas na área do varejo, mais precisamente empresas no segmento de *e-commerce*, onde é possível comprar praticamente qualquer coisa com apenas alguns cliques no mouse do computador, a entrega se torna uma das coisas mais importantes seja nos processos dentro de um armazém ou para a satisfação dos clientes, isso acontece em razão de quando é feito uma compra *online* o cliente não tem o produto imediatamente em mãos, portanto a experiência de compra é diferente de uma compra em loja física (BENITEZ, 2021). E o processo de *picking* está diretamente ligado com o processo de entrega, já que um atraso no *picking* poderá fazer o pedido não entrar no próximo horário de coleta da transportadora, e consequentemente atrasando a entrega.

1.2 Problemática

Visto que a entrega dentro do prazo é um dos maiores desafios em um *e-commerce* (CAVICCHIOLLI, 2021), como entregar uma boa experiência de compra? entregando os produtos dentro do prazo, em um mercado que está em constante crescimento. De acordo com um levantamento feito pela Worldpay from FIS, empresa com foco em produtos e serviços financeiros, o mercado do e-commerce no Brasil poderá ter um crescimento anual de aproximadamente 18% até 2025 (Juan Pablo D'Antiochia, 2022).

Assim otimizando o tempo dos processos realizados dentro de um armazém, poderá agilizar a entrega para o consumidor final. O gerenciamento de armazém é um dos processos logísticos com que uma empresa varejista deve se preocupar, já que tudo relacionado ao estoque trás impacto na entrega para o consumidor final, seja no tempo de entrega ou até mesmo na qualidade do produto. Nas várias operações de um armazém, a separação de pedidos é a mais demorada operação em geral (ROODBERGEN; KOSTER, 2001), por este motivo a separação de pedidos tem prioridade para a melhoria da produtividade (KOSTER; DUC; ROODBERGEN, 2007).

1.3 Objetivos

1.3.1 Objetivo geral

O objetivo deste trabalho é reduzir o tempo do processo de *picking* (separação de pedidos), em um layout de estoque baseado em um estoque real de uma empresa no segmento de *e-commerce*, com o uso de um algoritmo heurístico.

1.3.2 Objetivos específicos

- Determinação do layout;
- Implementar um algorítimo genético, para a roteirização de *picking*;
- Analisar resultados, em comparação com a estratégia de ordenação decrescente com base no endereço do produto.

1.4 Justificativa

A otimização da rota, priorizando minimizar a distância percorrida, tornará este processo mais rápido, agilizando o tempo de entrega para o consumidor final. É possível notar que este problema se trata de um problema de otimização, mais especificamente o problema do caixeiro viajante (HOFFMAN et al., 2013) problema pertencente a classe NP-Completo, este que não possui uma solução exata em tempo polinomial se o problema for grande, tentamos chegar o mais próximo dela possível por algoritmos heurísticos, por esse motivo será desenvolvido um algoritmo genético, que sua origem foi inspirada na teoria da evolução natural de Charles Darwin.

2 Revisão bibliográfica

Este capítulo apresenta os principais conceitos abordados no desenvolvimento deste trabalho, portanto auxiliando no entendimento dos capítulos posteriores.

Capítulo tendo inicio com uma breve introdução ao conceito de pesquisa operacional, área do conhecimento utilizada para a modelagem do problema de roteirização de *picking*. Em seguida uma seção para a abordagem do tema logística, principais áreas e processos. Também é apresentado o conceito de grafos, utilizado na modelagem do problema com pesquisa operacional, fazendo parte das restrições e presente no algoritmo, sendo o grafo a estrutura que representa o armazém.

E por fim três seções que estão interligadas, que são a seção de complexidade computacional, que explica as classes de complexidade de problemas, seção do problema do caixeiro viajante mostrando em qual classe de complexidade ele está incluído e o motivo desse problema para uma entrada grande ser solucionável apenas por algoritmos heurísticos, e finalizando com a seção do algoritmo genético, um algoritmo heurístico para solucionar o problema do caixeiro viajante.

2.1 Pesquisa operacional

Pesquisa operacional, tendo origem pelas pesquisas em operações militares no início da segunda guerra mundial, o objetivo era que cientistas ajudassem o exército em pesquisas para a definição de estratégias para o gerenciamento de recursos escassos para a realização de operações. Atualmente pesquisa operacional é utilizada para facilitar o entendimento e resolução de problemas muito complexos, problemas que geralmente possuem recursos limitados e restrições.

Este método de resolução de problemas é muito utilizado em organizações, por ser aplicável em áreas como manufatura, transporte, telecomunicações entre outras. A pesquisa operacional é realizada em partes, que são:

1. Coleta de dados, pegar toda informação necessária para a resolução do problema.

2. Construção do modelo científico, geralmente uma fórmula matemática, sendo uma representação suficientemente precisa do problema abordado, uma solução para este modelo também é válida para o problema real.

3. Validação do modelo, verificar se o resultado obtido pelo modelo é suficientemente bom, caso não seja será necessário o refinamento deste modelo (HILLIER; LIEBERMAN, 2013).

2.2 Logística

Logística, outra palavra com origem em atividades militares, usada para descrever gerenciamento de suprimentos, planejamento envolvendo tecnologias de guerra, e o uso do terreno ao seu favor (CAVALCANTE et al., s.d.), hoje a mesma palavra representa todos os processos de transporte e armazenamento dentro de uma empresa, da matéria prima ao produto acabado entregue para o consumidor final. Existem diferentes tipos de logísticas dentro de uma única empresa, os 4 principais tipos são:

- Logística de produção cuida de todo o processo de produção de um produto, a partir da matéria prima.
- Logística reversa é responsável pelo reaproveitamento dos produtos que voltam de um consumidor por algum motivo, é a mais responsável pela questão ambiental da empresa já que ela cuida do descarte consciente de resíduos.
- Logística de suprimentos, gerencia os recursos disponíveis como matéria prima que são materiais que serão transformados para a criação dos produtos, e também gerencia insumos, diferente da matéria prima são materiais usados para a criação dos produtos como máquinas e ferramentas.
- Logística de distribuição, é responsável pelo transporte do produto desde a fábrica até a casa do consumidor final (CONHEÇA, 2022).

Este trabalho pertence a logística de distribuição, cujo objetivo é otimizar o tempo de separação de pedidos, processo que pode interferir no tempo de entrega.

2.2.1 Processos de logística

Os processos logísticos do estoque podem ser classificados em três grupos, *inbound* (entrada de produtos), *outbound* (saída de produtos) e *warehouse* (gerenciamento de estoque), cada grupo tomando conta de uma parte da jornada do produto dentro da empresa, o trabalho focará em *outbound* por ser o grupo que a separação de pedidos pertence.

2.2.1.1 Separação de pedidos

O *Picking* (Separação de pedidos) consiste em o *picker* que é a pessoa designada a percorrer o estoque atrás dos produtos, com uma lista de separação em mãos, essa lista pode conter apenas produtos do mesmo pedido ou essa lista pode ser agrupada em lotes de separação, nesse caso terá diferentes pedidos e seguindo alguma regra de agrupamento por semelhança, como o tipo de pagamento ou o local que esses produtos

estão armazenados, a pessoa deve seguir uma rota de *picking* passando por todos os locais dos produtos contidos em sua lista.

Existem diferentes estratégias para montar a lista de separação de pedidos e facilitar o processo de acordo com o *layout* do centro de distribuição e os recursos disponíveis, que são:

- Separação discreta, estratégia mais simples de separação de pedidos, basicamente o *picker* irá pegar um pedido por vez e atendê-lo do começo ao fim.
- Separação por zona, neste modelo cada *picker* ficará responsável por uma zona do armazém, exemplo o *picker* 1 coleta todos os produtos que estão no corredor 10 e passará o pedido para o próximo *picker* que vai separar o resto dos produtos que estão no corredor 15, portanto neste modelo os pedidos sempre serão separados por mais de uma pessoa.
- Separação em onda, a separação de pedidos respeita uma "onda"que seria uma característica dos pedidos, neste caso o *picker* irá separar todos os pedidos que possuem uma característica em comum, como por exemplo pedidos feito para uma mesma região, cliente, ou até mesmo tipo de pagamento.
- Separação de pedidos por lote que é o modelo que será utilizado neste trabalho, os pedidos são agrupados em lotes, podendo ser criados pelo próprio *picker* ou de forma automática seguindo algum tipo de regra de criação, neste modelo serão separados vários pedidos de uma vez, todos que estão dentro do lote.

Após o fim da separação, precisa ser feito mais um passo chamado consolidação para separar os produtos de cada pedido individualmente (TIPOS, 2017).

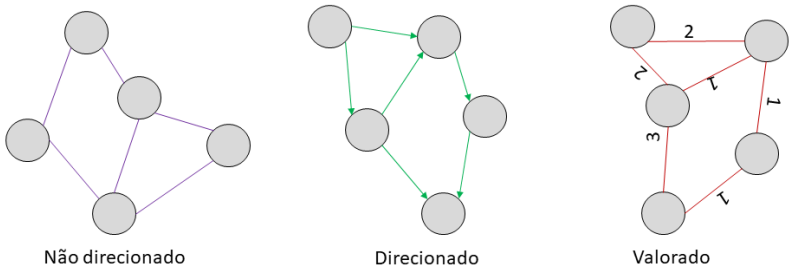
2.3 Grafos

Grafo é uma estrutura de dados, e pode ser representado por G = (V,E), onde V é um conjunto discreto de pontos e E é uma família de elementos não vazios, os elementos do conjunto V são chamados de vértices, nós ou pontos, um vértice é adjacente a outro se estiverem ligados por uma conexão de adjacência que são os elementos de E, tais elementos podem ser chamados de arestas em estruturas não orientadas e arcos nas orientadas, uma aresta pode ser associada a um valor numérico, chamado de peso. Existem algumas categorias de grafos limitadas pelo uso de suas arestas, que são:

- Grafo direcionado, grafo onde uma aresta de um vértice está direcionado para outro vértice, portanto um caminho dentro de um grafo direcionado será válido apenas se existir uma aresta partindo do vértice de origem direcionado para o vértice de destino e o caminho inverso, só é possível se houver outra aresta direcionada no sentido contrário.

- Grafo não direcionado, nessa categoria o caminho pode ser feito em qualquer direção desde que exista uma aresta ligando os dois pontos.
- Grafo valorado, um grafo é valorado se suas arestas possuem um peso, valor atribuído as arestas para representar alguma especificidade do problema, como o tempo de percurso de um vértice até outro, distância, custo, ou qualquer outra coisa que ajude a estruturar o problema, um grafo direcionado ou não direcionado também podem ser valorados (NETTO, 2003). Abaixo na figura 2.1 uma ilustração das categorias de grafos.

Figura 2.1 – Grafos



Fonte: Criado pelo autor

2.4 Complexidade Computacional

Nesta seção será introduzido complexidade computacional, classes de complexidade e o que faz um algoritmo ser razoavelmente rápido. Mesmo um problema sendo decidível, portanto computacionalmente solúvel, na prática dependendo do consumo de memória e tempo ele pode acabar se tornando não solúvel.

Classes de complexidades de problemas, são conjuntos de problemas relacionados aos recursos computacionais necessários para encontrar uma solução.

Classe P contém os problemas de decisão que possuem algoritmos de tempo polinomial, dado uma instância do problema existe um algoritmo que em tempo polinomial consegue retornar uma solução, algoritmos com tempo polinomial são considerados razoavelmente rápidos.

Classe NP, diferente da classe P, ainda não foi encontrado nenhum algoritmo que consiga retornar uma solução em tempo polinomial para uma instância do problema, porém com um certificado (solução do problema), podemos passá-lo para um verificador, que irá retornar "aceito"ou "rejeito", esse verificador irá retornar aceito restritamente em tempo polinomial, mas não há limite superior de tempo para o retorno "rejeito", o verificador é um algoritmo que verifica uma possível solução correta.

Classe NP-completo, existe um subconjunto em NP, que são os problemas NP-completos, para um problema A ser NP-completo ele deve estar em NP e qualquer problema B em NP deve ser redutível a A, B é redutível a A se para qualquer instância de B pode ser construído uma instancia de A em tempo polinomial, quando um problema B se reduz a A uma solução para o problema A pode ser usada para resolver B.

Classe NP-Difícil, classe de problemas de otimização, não existe nenhum algoritmo que em tempo polinomial encontre a solução ótima, é usado algoritmos heurísticos, para chegar o mais próximo da solução ótima possível, na prática essa solução pode ser suficientemente boa e mais fácil de encontrar.(SIPSER, 2005)

2.4.1 Problema do Caixeiro Viajante

Problema de otimização combinatória, cujo objetivo original é, dado um conjunto de cidades e o custo de viagem até cada uma delas, deve-se encontrar o menor percurso passando por todas as cidades e retornando para o local de origem, parece um problema simples, porém vem sendo assunto de pesquisas em diversas áreas do conhecimento, por ser um problema aplicável em áreas como genética, telecomunicação, neurociência e logística (APPLEGATE et al., 2007).

Outro motivo que torna este problema tão famoso é sua complexidade computacional, até o momento não foi encontrado nenhum algoritmo que consiga retornar uma solução para o problema em tempo polinomial, este é um problema intratável computacionalmente quando tentamos encontrar uma solução ótima, por se tratar de um problema NP-completo, como foi mostrado na seção anterior, um problema A é NP-completo se para qualquer problema B em NP, pode ser redutível a A, desta forma se for encontrado um algoritmo que resolva o problema do caixeiro viajante em tempo polinomial, é provado que NP=P, isto vale para qualquer problema em NP, tal prova que vem sendo uma dúvida para muitos pesquisadores da área (FORTNOW, 2009).

2.5 Algoritmos Genéticos

Algoritmos genéticos são algoritmos inspirados na teoria da evolução por seleção natural, proposto por Charles Darwin, teoria em que os mais aptos sobrevivem. As diferenças dos algoritmos genéticos quando comparados com outros métodos de otimização tradicionais são:

- Algoritmos genéticos trabalham com uma codificação do conjunto de parâmetros, não com os próprios parâmetros.
- Algoritmos genéticos trabalham com uma população de pontos, e não com um único ponto, isso cria uma diversificação nos dados, diminuindo as chances de encontrar máximos locais.

- Algoritmos genéticos utilizam recompensas (*objective function* ou *fitness function*) e não derivada ou outro conhecimento auxiliar.

- Algoritmos genéticos utilizam regras de transição probabilísticas, não determinísticas (GOLDBERG, 1988).

2.5.1 Funcionamento do algoritmo

Em algoritmos genéticos, uma possível solução correta é chamada de indivíduo, e este indivíduo é constituído de genes, que é o domínio do problema, um conjunto de indivíduos é chamado de população, e esses indivíduos são submetidos a algumas operações para ser encontrado uma solução cada vez mais próxima da solução ótima, e os ciclos de operações realizadas nos indivíduos contidos na população, é chamado de geração, geralmente são realizadas 5 operações, uma para inicialização da população e 4 que vão modificando os indivíduos da população.

Inicialização, a primeira população geralmente é criada aleatoriamente, isso pode resultar em bons indivíduos logo no início, assim tornando a evolução nas próximas gerações mais rápida.

Avaliação de aptidão, é a função chamada de *objective function* ou *fitness function*, o objetivo dela é calcular a aptidão de todos os indivíduos dentro da população, isso de acordo com o problema a ser solucionado, portanto para cada problema resolvido por um algoritmo genético, existe uma *fitness function* diferente.

Seleção, aqui é feito a escolha dos indivíduos que serão submetidos a uma operação de combinação, os indivíduos que obtiveram valores de aptidão mais elevados na operação anterior, possui mais chances de serem escolhidos nesta etapa (GOLDBERG, 1988), existem alguns métodos para a escolha dos indivíduos, alguns desses métodos são:

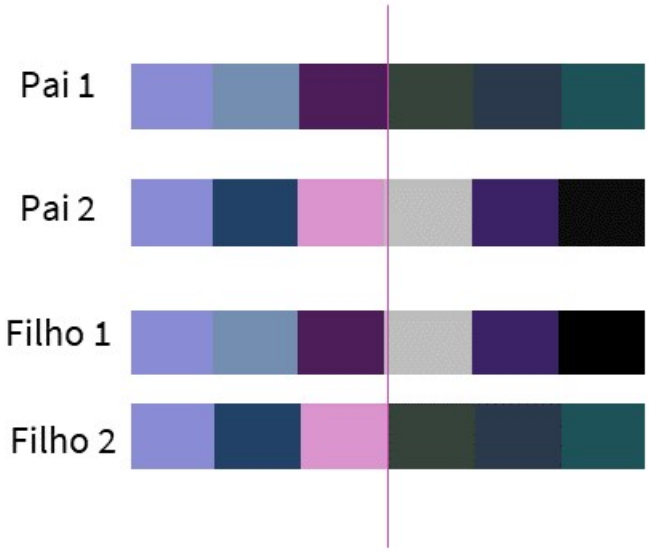
- Seleção por torneio, um grupo de indivíduos é escolhido aleatoriamente e o indivíduo com maior aptidão é selecionado.

- Seleção por truncamento, dado um valor x serão escolhidos os x mais aptos indivíduos da população.

- Seleção por normalização linear, aqui os indivíduos são ordenados de acordo com o valor de aptidão, após a ordenação são trocados os valores de aptidão com base na localização, o indivíduo com maior aptidão recebe o valor máximo, e o com menor aptidão o valor mínimo, e é distribuído linearmente os outros valores entre o mínimo e o máximo para os demais indivíduos, assim mesmo que o indivíduo com maior aptidão estiver muito acima dos demais, a diferença de probabilidade de serem escolhidos não é tão grande, tornando a população mais diversificada (PACHECO et al., 1999).

Após a seleção é executado as funções de *crossover* e mutação, primeiro vem o *crossover*, operação aplicada em um par de indivíduos gerando dois indivíduos filhos, uma maneira de gerar estes novos indivíduos é, dividi-los em uma posição aleatória, obtendo duas cabeças e duas caudas, e trocar as caudas, gerando os filhos como mostrado na figura 2.2.

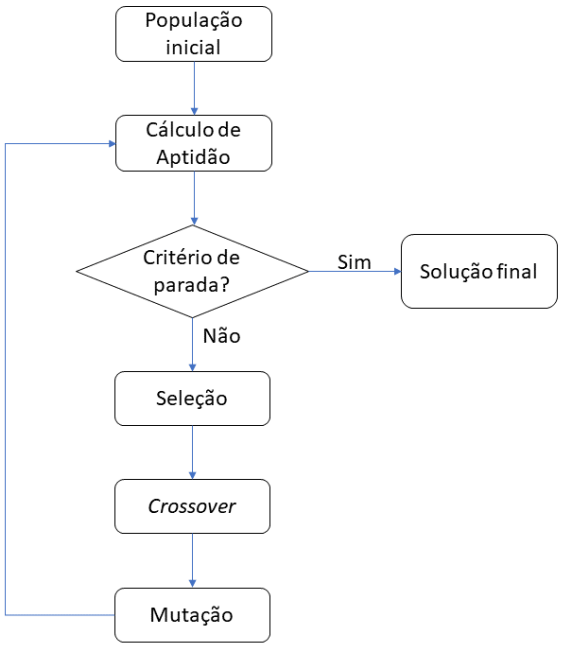
Figura 2.2 – Crossover



Fonte: Criado pelo autor

Para que algumas soluções sejam preservadas, o *crossover* é realizado dada uma probabilidade, assim nem todos os pares de indivíduos escolhidos vão fazer o *crossover*, então os filhos serão exatamente iguais aos pais. Após o *crossover*, também dada uma probabilidade, acontece a mutação, nos indivíduos filhos, que são operações realizadas nos genes, um exemplo seria trocar um gene de posição, ou trocar o seu valor, mas após essa operação, a solução anterior deixará de existir, que poderia ser uma boa solução, por esse motivo é necessário um ponderamento na escolha da taxa de mutação de uma implementação de algoritmo genético, com uma taxa muito alta o algoritmo torna sua busca aleatória, uma taxa muito baixa aumentará o tempo de conversão para a solução final, portanto é necessário a realização de testes na hora da escolha da taxa de mutação (LACERDA; CARVALHO, 1999). O algoritmo não termina, até encontrar uma solução satisfatória, o critério de parada deve ser fornecido pelo desenvolvedor do algoritmo. A figura 2.3 é um fluxograma do funcionamento do algoritmo genético, mostrando desde a inicialização da população, as operações que fazem parte de uma geração até o critério de parada ser cumprido, assim retornando a melhor solução da população atual.

Figura 2.3 – Fluxograma Algoritmo genético



Fonte: Criado pelo autor

2.5.2 Aplicações

Após essa breve explicação sobre algoritmos genéticos, aqui dois exemplos de problemas solucionados e projetos desenvolvidos em áreas distintas com algoritmos genéticos.

- Música Evolutiva: Uma Abordagem Computacional para Composição Algorítmica, aqui um exemplo de criação de melodia por meio de algoritmos genéticos (FREITAS, 2011).
- Neural Networks using Genetic Algorithms, outra aplicação bastante interessante de algoritmos genéticos, é o treino de uma rede neural, neste trabalho é dado exemplos de como é possível treinar uma rede neural com um algoritmo genético, um dos exemplos seria cada indivíduo ser formado de strings representando pesos de uma rede completa, e o fitness ser o cálculo do erro (alvo - saída) (MAHAJAN; KAUR, 2013).

As aplicações de algoritmos genéticos vão muito além dos exemplos citados, muitas áreas do conhecimento pode se beneficiar com o seu uso.

2.6 Trabalhos Relacionados

Seção cujo objetivo é apresentar outros trabalhos que possuem semelhanças com o trabalho atual, e analisar essas semelhanças. Foram pesquisados dois trabalhos bem diferentes, porém ao mesmo tempo com o mesmo objetivo, a otimização do processo de *picking* de pedidos.

2.6.1 *An algorithm for dynamic order-picking in warehouse operations*

O objetivo em *An algorithm for dynamic order-picking in warehouse operations* por Wenrong Lu, Duncan Mcfarlane, Vaggelis Giannikas, e Quan Zhang (2015), foi otimizar o tempo de separação de pedidos, com um algoritmo de roteamento intervencionista, em um modelo de ordenação de *picking* dinâmico, este modelo é como uma criação de lista de separação em lotes explicado em seções anteriores, porém ao invés de ser uma lista estática em que o *picker* não irá pegar nenhum outro pedido que não esteja em sua lista até terminar de separar todos os pedidos, sua lista esta sujeita a mudanças enquanto ele está no processo de separação, portanto se no momento que ele estiver em busca dos produtos aparecer um pedido urgente, este pedido pode ser acrescentado em sua lista, no modelo estático este pedido seria alocado em outra lista de separação não inicializada, até que algum *picker* fique disponível para dar inicio a ela (LU et al., 2015).

O algoritmo proposto diferente do algoritmo genético, retorna uma solução ótima em tempo polinomial, porém isso só é possível porque o layout do estoque utilizado no trabalho possui menos de três blocos (zonas), o algoritmo foi desenvolvido com base no algoritmo de otimização de Ratliff e Rosenthal (1983), mas foram criadas algumas extensões neste algoritmo base para suportar a interrompibilidade, e iniciar o *picking* em qualquer lugar do armazém. No desenvolvimento do trabalho foram determinados alguns critérios que um algoritmo IOP (*Interventionist Order-Picking*) deveria cumprir, que são:

- Interruptibilidade, o algoritmo deve suportar a lista sendo interrompida enquanto o processo de *picking* estiver em andamento, conseguindo calcular a menor rota, agora considerando o novo pedido adicionado na lista.
- Otimização, o algoritmo deve retornar a menor rota, de acordo com os itens contidos na lista.
- Complexidade, o tempo para o cálculo da rota deve ser polinomial e linear de acordo com o número de corredores presentes na coleta.
- o selecionador sempre iniciará a lista e irá finalizá-la no mesmo local, isso já pré-definido (LU et al., 2015).

Além disso, o trabalho analisado possui muitas semelhanças com o projeto abordado neste trabalho de conclusão de curso, os dois compartilham do mesmo objetivo que é otimizar o processo de separação de pedidos em um armazém, os dois propõem a criação de um algoritmo de roteirização, mas nesta parte que começa surgir as diferenças, o trabalho abordado por Wenrong Lu, Duncan Mcfarlane, Vaggelis Giannikas, e Quan Zhang, tem como problema a roteirização de um processo de separação de pedidos dinâmico, já neste projeto será um processo estático, no trabalho é encontrado uma solução em tempo polinomial, já no *layout* de estoque utilizado neste projeto até o momento não é possível a criação de um algoritmo com tempo polinomial, por isso será utilizado um algoritmo heurístico, mais especificamente um algoritmo genético.

2.6.2 Proposta de um modelo heurístico para a roteirização de empilhadeiras em um armazém de grande porte

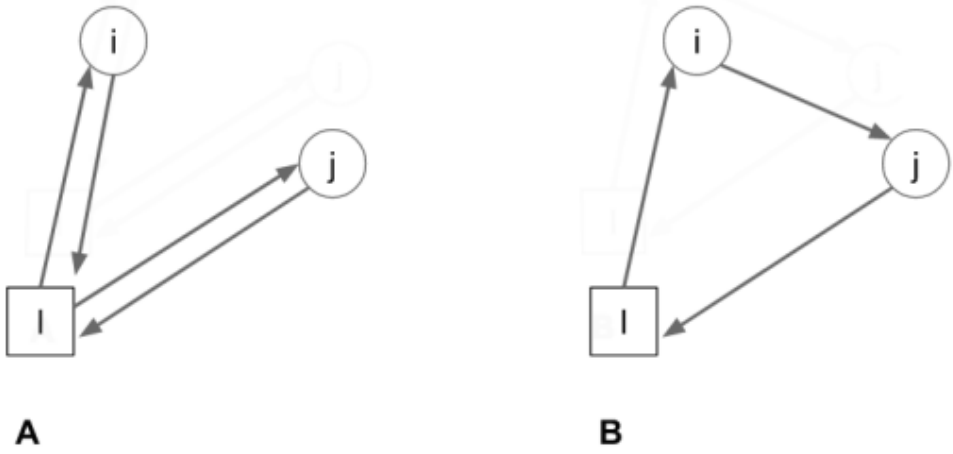
Em Proposta de um modelo heurístico para a roteirização de empilhadeiras em um armazém de grande porte, desenvolvido por Maiko Andrei Nienkotter (2017), é proposto um modelo heurístico para a roteirização de *picking* e armazenamento, utilizando empilhadeiras, foi feito um estudo de caso e foi observado que as empilhadeiras ficam ociosas em determinados intervalos de tempo. No processo de *picking* a empilhadeira vai até o local de carregamento sem produtos, faz a busca dos produtos na lista de separação e retorna com os produtos, no processo de armazenagem a empilhadeira vai com produtos e retorna sem, analisando estes processos foi verificado que seria possível combiná-los compartilhando o uso das empilhadeiras, reorganizando as rotas (NIENKOTTER et al., 2017).

O trabalho teve como inspiração o algoritmo de Clark & Wright, que também propõe uma reorganização de rotas, quando elas compartilham o mesmo local de origem. A entrada do algoritmo são OTs (ordens de transferências) que no armazenamento são geradas a partir do momento que os códigos de barras dos produtos são escaneados para identificar a entrada no armazém, essas OTs são listas dos produtos recebidos para armazenagem no estoque, no *picking* essas OTs são emitidas no momento da venda, assim também criando o trajeto de entrega, e o agendamento da retirada da carga pela transportadora, que devem estar obrigatoriamente 20 minutos com antecedência no local de coleta, para o carregamento do caminhão, diferente da armazenagem que as OTs são listas de armazenagem, no *picking* são listas de saída (NIENKOTTER et al., 2017). As OTs são agrupadas para a análise da rota com base em uma janela de tempo, que é o tempo restante até a coleta pela transportadora.

Neste algoritmo heurístico proposto por Maiko Andrei Nienkotter, em cada iteração é verificado a economia na distância percorrida, e o cálculo deste ganho se da pela diferença entre o custo total da rota atual A com o custo total da rota proposta reorganizada

B, utilizando algoritmo de Dijkstra para calcular a distância entre dois pontos em um grafo. A figura 2.4 ilustra como seria a rota A sem a reorganização e a rota B interligando os dois processos.

Figura 2.4 – Exemplo do Método de Clark & Wright



Fonte: Nienkotter et al. (2017)

Porém o tempo de computação do algoritmo após o acréscimo do cálculo das distâncias subiu consideravelmente, portanto foi necessário que este cálculo fosse feito previamente, com todas as combinações de pontos no grafo dois a dois, assim armazenando essas distâncias em um arquivo para apenas serem consumidas pelo algoritmo heurístico posteriormente.

Após a análise do trabalho, é possível identificar algumas semelhanças e diferenças com o projeto atual, Maiko Andrei Nienkotter também tem como objetivo otimizar o processo de *picking* dentro de um armazém, entretanto seu foco não é encontrar o caminho mais curto e sim otimizar o processo com um melhor aproveitamento de tempo, que no caso seria o tempo em que uma empilhadeira está se locomovendo até seu objetivo, porém sem realizar alguma tarefa, outro ponto de semelhança é o tipo de algoritmo utilizado, que nos dois trabalhos é implementado um algoritmo heurístico, mas com entradas diferentes, no trabalho analisado a entrada para o algoritmo retornar a solução são ordens de transferências, já no trabalho atual serão produtos.

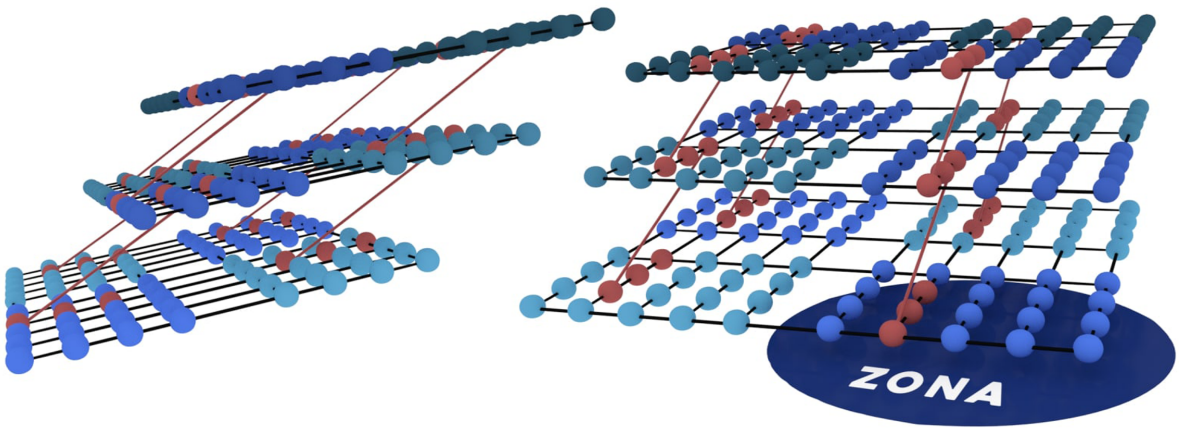
3 Desenvolvimento

Capítulo apresentando todas as etapas de desenvolvimento do trabalho, a modelagem das restrições transformando o layout do armazém em um grafo não direcionado, o modelo matemático, e por fim o desenvolvimento do algoritmo genético para minimizar a distância da rota de picking em comparação com uma rota gerada sem um algoritmo.

3.1 Elaboração do modelo de grafo

O armazém abordado neste trabalho possui 3 andares, 24 zonas e 4 escadas por andar e foi desenvolvido utilizando o conceito de grafos, onde cada estante é um vértice, porém em um corredor dentro de um armazém real existem estantes dos dois lados do corredor, portanto neste modelo cada dupla de estantes (lado direito e lado esquerdo) será representado por apenas um vértice, visto que um picker estando em uma posição dentro do corredor é indiferente ele pegar o produto na estante a sua direita ou a sua esquerda, pelo motivo de o corredor ser estreito, e cada um desses vértices estão ligados com outras estantes por arestas formando os corredores, zonas e andares, as ligações entre andares se da por meio de vértices que representam as escadas. Todas essas ligações possuem um peso que é a distância entre dois vértices, neste modelo existem 4 valores de pesos, peso 1 para arestas dentro do mesmo corredor, 2 para arestas que ligam um corredor a outro, 3 em arestas entre zonas e 4 de peso para as arestas entre andares. A figura 3.1 é um exemplo reduzido do grafo do armazém, com apenas 12 zonas, e quantidade reduzida de estantes por corredor, os vértices e arestas em vermelho representam as escadas e suas conexões.

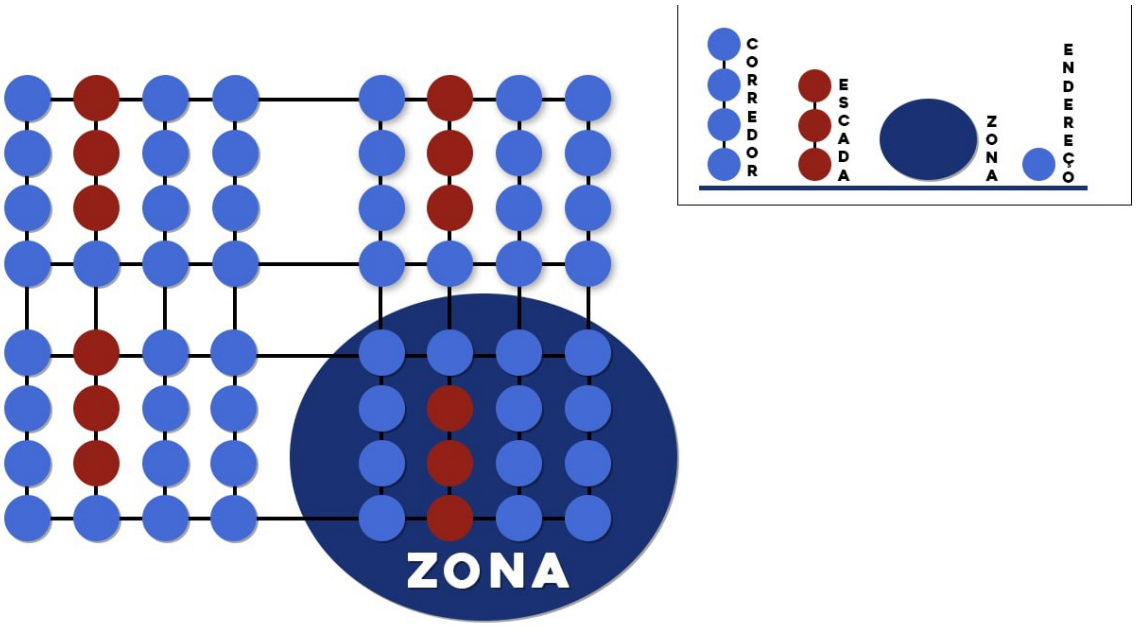
Figura 3.1 – Grafo Armazém



Fonte: Criado pelo autor

A imagem 3.2 é apenas o primeiro andar visto de cima para melhor visualização.

Figura 3.2 – Grafo visualização superior

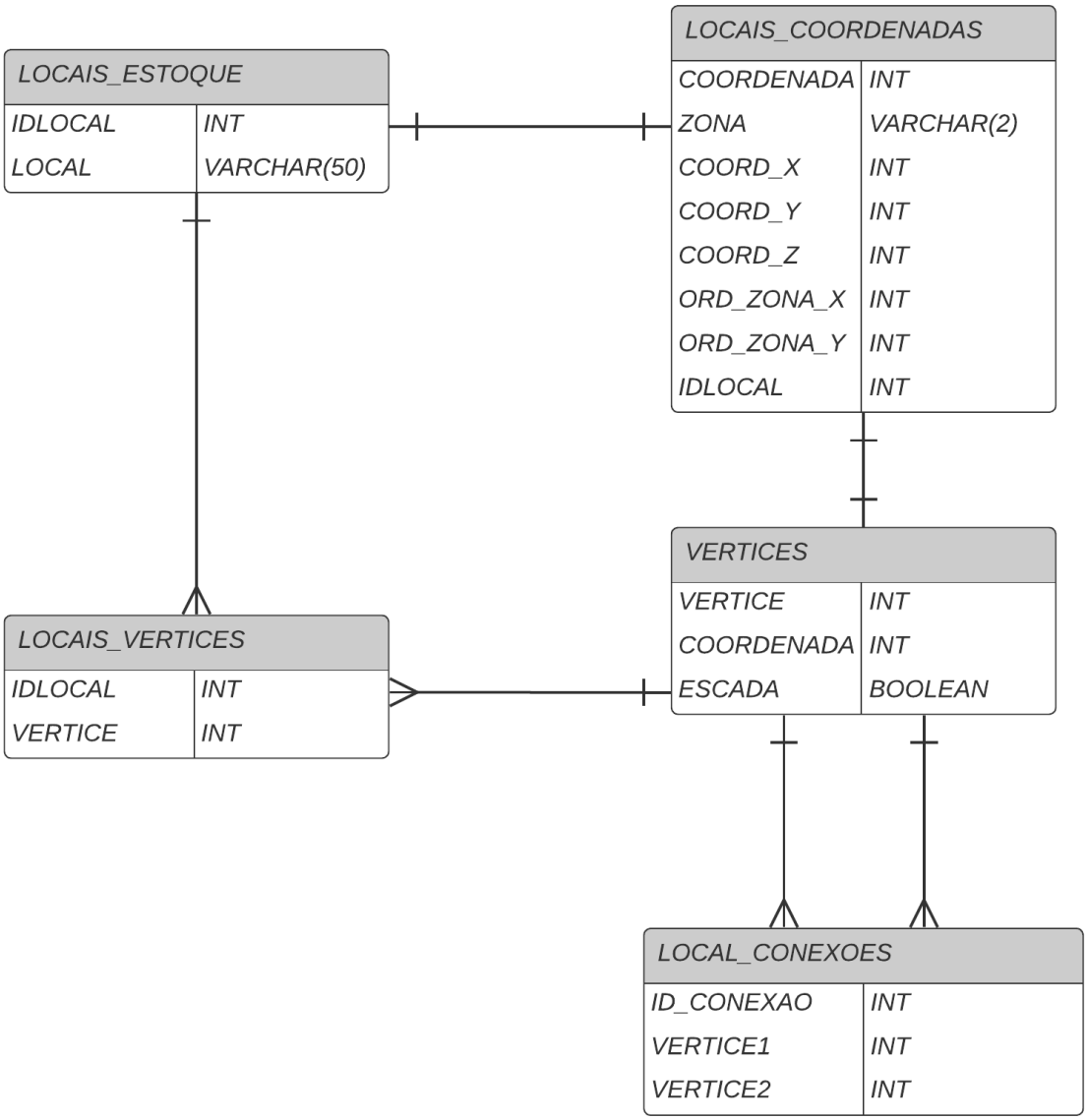


Fonte: Criado pelo autor

Este modelo foi desenvolvido utilizando querys sql, e a biblioteca iGraph da linguagem python, biblioteca open source utilizada para manipulação e análise de grafos. Primeiramente foi criado tabelas em um banco de dados SQL Server para armazenar os locais de estoque, coordenadas desses locais, vértices representando cada local, as conexões entre os vértices com base nas coordenadas, e uma tabela representando a fila de lotes aguardando o picking.

As conexões entre os vértices foi realizada por meio de consultas sql, conectando locais adjacentes e os andares, e é possível verificar se um local é adjacente a outro por meio de seu endereço armazenado na tabela locais_estoque, esses endereços seguem um padrão, exemplo de endereço: 2.A1.06.16.03.02, o endereço possui 6 informações, primeiro vem o andar (2), segundo a zona (A1), terceiro o corredor (06), quarto estante dentro do corredor (16), quinto nível na estante (03), e por fim é um número que varia de 1 a 2 que representa o bin (02), que são caixas e neste caso o produto estaria na segunda caixa. Após a criação de todas as conexões, o código python consome a tabela local_conexoes passando os vertice1 e os vertice2 para uma função da biblioteca iGraph criar o grafo a ser utilizado no algoritmo.

Figura 3.3 – Diagrama tabelas



Fonte: Criado pelo autor

3.2 Elaboração do modelo Matemático

O modelo matemático do problema de roteirização de picking, será basicamente o mesmo modelo do problema do caixeiro viajante, que pode ser formulado como um programa linear inteiro, neste caso será utilizado a formulação de Miller–Tucker–Zemlin (MILLER; TUCKER; ZEMLIN, 1960).

$$X_{ij} = \begin{cases} 1, & \text{se a rota incluir uma ligação direta entre os locais i e j} \\ 0, & \text{caso contrário.} \end{cases}$$

$i, j \in \{1,2,...,n\}$

Função objetivo:

$$\min \sum_{i=1}^{n} \sum_{j=1 j \neq i}^{n} C_{ij} X_{ij} \tag{3.1}$$

Onde C_{ij} é o peso da aresta entre o vértice i e o vértice j.

Restrições: Restrição de entrada. Cada local deve ser visitado a partir de um único local:

$$\sum_{i=1 i \neq j}^{n} X_{ij} = 1, j \in \{1,2,...,n\} \tag{3.2}$$

Restrição de saída. A partir de um local é possível visitar apenas um próximo local:

$$\sum_{j=1 j \neq i}^{n} X_{ij} = 1, i \in \{1,2,...,n\} \tag{3.3}$$

Restrições de subrotas:

$$\begin{aligned} u_i - u_j + n x_{ij} \leq n - 1,\ i,j \in \{1,2,...,n\}, i \neq j, \\ 2 \leq u_i \leq n\ i \in \{2,...,n\} \\ u_i \in \mathbb{R}^+, i \in \{2,...,n\}, u_1 = 1 \end{aligned} \tag{3.4}$$

3.3 Implementação do algoritmo genético para roteirização

Após toda a modelagem do armazém em um grafo, foi desenvolvido o algoritmo genético para a otimização da rota. Antes da execução do algoritmo é verificado se existe um arquivo no projeto, que contém todas as distâncias do grafo de um vértice i a um vértice j, de todos os possíveis pares de vértices, se o arquivo não existir é chamado o algoritmo desenvolvido para o cálculo dessas distâncias e a armazenagem em um arquivo csv, caso exista só é armazenado essas distâncias em uma estrutura de dicionário do python, para serem consumidas posteriormente pelo algoritmo. Em seguida é realizado a busca pelos lotes de separação no banco de dados, retornando todos os produtos contidos em cada lote e seus respectivos endereços.

Ainda antes da execução do método principal, é enviado a lista de lotes para um método que analisa endereços duplicados, pelo motivo que para o algoritmo de roteirização é importante o endereço apenas até a informação da prateleira, portanto se houver dois produtos em um lote na mesma prateleira, mas em níveis diferentes, os dois produtos serão ligados ao mesmo identificador de vértice gerando duplicidade, então nesses casos o analisador de duplicidade gera uma novo vértice auxiliar para o endereço duplicado, copiando as conexões do vértice original, para que os dois produtos sejam representados por vértices diferentes.

Terminando todas as verificações e tendo todos os dados necessários para o funcionamento do algoritmo genético, é realizado sua execução, recebendo 5 parâmetros, sendo eles:

- O grafo do armazém
- A rota inicial sendo a rota gerada sem algoritmo, apenas ordenando a busca pelos endereços de forma decrescente
- O número identificador do lote
- Dicionário com as distâncias do arquivo csv
- O número de indivíduos que serão escolhidos para o crossover

O algoritmo inicia gerando a população inicial aleatória, o número de indivíduos da população é 1 + o número máximo de combinações dois a dois com base no número de pais que serão selecionados na etapa de seleção, esse 1 a mais é a rota inicial que também é necessário estar na população para não correr o risco de o algoritmo retornar uma solução pior que a rota inicial. Cada indivíduo da população é um array que representa uma rota, e cada índice do array são os genes, vértices que representam os locais que os produtos do lote estão armazenados.

Após a geração da população inicial o algoritmo entra no laço de repetição que realiza as operações que fazem parte das gerações, o critério de parada para o retorno da melhor solução é o passar de 200 gerações sem nenhuma evolução nos indivíduos, saindo do laço e retornando a melhor solução encontrada. No decorrer das gerações (repetições no laço), são realizadas as operações de seleção, *crossover* e mutação introduzidas na seção 2.5.

A primeira operação realizada é a de seleção, será utilizado o método de seleção por truncamento que seleciona os n indivíduos mais aptos da população atual, n é o número de indivíduos que serão escolhidos para o *crossover* fornecido no inicio do algoritmo. A escolha dos mais aptos é feita com base no *fitness* de cada indivíduo, também conhecido como valor de aptidão, valor calculado no método *fitness* desenvolvido, o cálculo do *fitness* é dado pelo seguinte somatório:

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n} D_{ij} \tag{3.5}$$

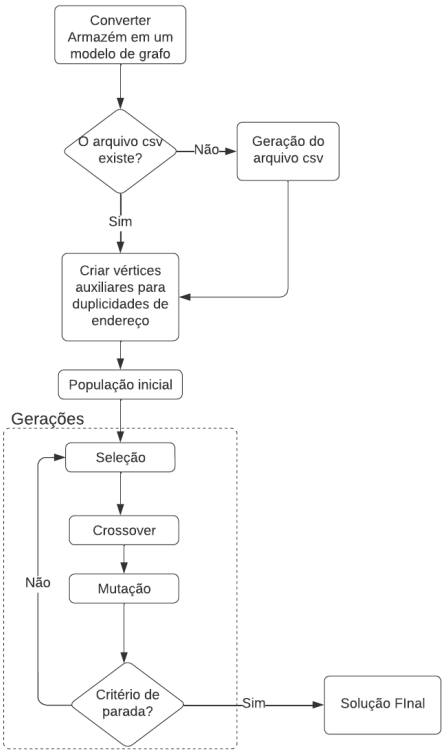
Assim serão somadas todas as distâncias do vértice i para o vértice i+1 até o percorrer todo o vetor do indivíduo, obtendo a distância da rota e retornando como seu valor de *fitness*, multiplicado por -1, para minimizar a distância e maximizar o `fitness`. O cálculo da distância do vértice i para o vértice i+1 não é necessário, visto que ele já foi realizado anteriormente e salvo no arquivo csv que foi transformado em um dicionário, desse modo só será necessário encontrar a distância i para i+1, procurando pela chave sendo os dois vértices concatenados.

Em seguida da operação de seleção será realizada a operação de *crossover*, nos indivíduos selecionados, que tipicamente são chamados de "pais", mas antes de qualquer operação ser realizada, são escolhidos n pais que não irão realizar o crossover, isso para manter a diversidade da população, n tendo que ser previamente fornecido no algoritmo. Já no crossover é escolhido um número x aleatório estando entre 0 e o tamanho do indivíduo, x será o ponto de corte nos genes dos pais, tendo o valor de x é realizado o corte nos dois indivíduos na posição x, gerando duas cabeças e duas caudas e trocando as caudas dos pais, assim gerando dois novos indivíduos filhos, será realizado essa operação em todas as combinações dois a dois dos indivíduos pais, assim gerando uma nova população, por esse motivo que o tamanho da população inicial é 1 + o número máximo de combinações dois a dois com base no número de pais, para após o *crossover*, a população continuar com o mesmo tamanho.

A última operação realizada em uma geração é a mutação, que neste caso diferente do *crossover* que é escolhido um número n de indivíduos para não serem submetidos as alterações, será escolhido uma taxa de mutação, que é a probabilidade de um indivíduo sofrer a mutação. Na mutação é escolhido dois números aleatórios x e y com o valor entre 0 e o tamanho do indivíduo, após a escolha dos números é trocado o valor do gene no índice x pelo gene no índice y.

Terminando todas as operações presentes na geração, é calculado o *fitness* do individuo mais apto até o momento, para comparação com o mais apto da próxima geração até que passe 200 gerações sem nenhuma evolução, retornando a melhor rota até o momento.

Figura 3.4 – Fluxograma Algoritmo roteirização de picking



Fonte: Criado pelo autor

3.4 Resultados

Foi executado o algoritmo para 300 lotes de separação gerados aleatoriamente, com tamanhos variados, comparando o resultado da rota gerada pelo algoritmo e a rota gerada sem algoritmo apenas ordenando a busca dos produtos de forma decrescente com base nos endereços de armazenagem, e após a execução de cada lote são armazenados os resultados em um arquivo csv. Tendo os dados da execução dos 300 lotes, foi realizada a importação do arquivo em uma tabela do banco de dados sql para facilitar a análise.

Figura 3.5 – Médias e ganhos máximo e mínimo

	Média de ganho (%)	Média de tempo de execução	Ganho máximo (%)	Ganho mínimo (%)
1	17,2546792604213	17,3201431888545	45,6730769230769	0

Fonte: Criado pelo autor

A imagem 3.5 mostra o resultado de uma *query* sql em cima dos resultados importados do csv, retornando 4 dados importantes para a análise. A média de ganho, portanto é otimizado em média 17,25% da rota com o algoritmo de roteirização, sendo assim dois *pickers* separando o mesmo lote o que irá percorrer a rota proposta pelo algoritmo irá andar 17,25% a menos que o picker que irá percorrer a rota com a estratégia decrescente. Média de tempo de execução, o algoritmo demora em média 17,32 segundos para retornar uma rota para um lote, esse tempo é proporcional ao tamanho do lote, é possível verificar isso com o gráfico de dispersão na figura 3.6.

Figura 3.6 – Média de tempo por quantidade de produtos

Fonte: Criado pelo autor

O ganho máximo para a execução dos 300 lotes foi de 45,67%, já o ganho mínimo foi de 0,0%. Apenas 1 lote dos 300 executados o algoritmo não conseguiu otimizar, analisando os endereços dos produtos dentro dos dois lotes o lote com ganho máximo e o lote com ganho mínimo, é possível perceber que os produtos do lote com ganho máximo estão mais espalhados pelas zonas e andares.

Figura 3.7 – Lotes ganho máximo e mínimo

Lote ganho mínimo

LOTE	PRODUTO	ENDEREÇO
178	Produto6	1.A1.03.21.05.02
178	Produto5	1.A1.05.07.04.02
178	Produto2	2.A1.02.02.06.02
178	Produto1	2.A1.07.12.01.04
178	Produto3	2.B2.01.04.03.05
178	Produto4	2.B2.05.16.02.02

Lote ganho máximo

LOTE	PRODUTO	ENDEREÇO
2	Produto8	1.C2.04.25.05.05
2	Produto10	2.A2.01.01.05.01
2	Produto5	2.A2.02.05.02.01
2	Produto3	2.A2.05.03.01.02
2	Produto1	2.B1.02.07.06.02
2	Produto4	2.B1.08.03.02.02
2	Produto6	2.C1.02.06.01.05
2	Produto2	2.C2.07.03.04.04
2	Produto9	3.A1.03.01.03.02
2	Produto7	3.A2.05.26.03.01

Fonte: Criado pelo autor

Analisando a figura 3.7 apesar da diferença de tamanho o lote 178 possui os produtos muito próximos um do outro, os produtos estão espalhados no andar 1 e 2, porém no andar 1 os produtos estão apenas na zona A1, no andar 2 estão na zona A1 e B2 porém 2 produtos em cada zona. Já no lote 2 os produtos estão espalhados nos 3 andares e com uma diversidade de zonas maior.

O algoritmo só irá conseguir otimizar a rota se os produtos estiverem bem espalhados, se já estiverem próximos a estratégia decrescente com base no endereço já traz um bom resultado.

4 Conclusão e trabalhos futuros

A roteirização de *picking* utilizando algoritmo genético se mostrou eficiente para minimizar a distância percorrida pelo *picker*, nem sempre o algoritmo irá retornar a melhor solução, porém para armazéns que não utilizam nenhum algoritmo para a geração da rota, apenas estratégias como a ordenação decrescente com base no endereço, conseguirá ter uma melhora significativa no tempo de *picking* dependendo do layout do armazém e seu tamanho, pelo motivo de se o armazém for de pequeno porte os produtos já estarão próximos, assim impossibilitando a otimização com o algoritmo, como visto na seção 3.4.

Mesmo o algoritmo tendo se mostrado eficiente para a roteirização, o seu tempo de execução aumenta consideravelmente com o aumento da quantidade de produtos no lote, dependendo do valor da taxa de entrada de pedidos para um armazém e o tamanho dos lotes, o tempo para o algoritmo otimizar a rota pode acabar inviabilizando o seu uso, portanto para o seu uso deve haver uma ponderação na hora da escolha do tamanho dos lotes, para ficarem com um tamanho mediano, lotes pequenos não seria necessário o uso do algoritmo genético, já que seria possível desenvolver um algoritmo determinístico que otimize a rota em tempo polinomial.

Em trabalhos futuros, seria interessante o desenvolvimento do algoritmo de roteirização na criação dos lotes, os lotes já seriam criados com base na melhor rota e não apenas reorganizado a ordem de busca dos produtos, isso iria extrair mais da capacidade que algoritmos genéticos possuem. Outro desenvolvimento a parte seria integrar o algoritmo de roteirização de picking com um programa de navegação *indoor*, assim ao gerar a rota o programa iria retornar visualmente como um gps por onde o *picker* deverá passar até chegar no próximo local de busca. O algoritmo atual retorna todos os locais por onde o *picker* terá que passar, porém não foi desenvolvido algo que mostre essa informação visualmente para o *picker*.

Referências

APPLEGATE, D. L. et al. *The Traveling Salesman Problem: A Computational Study*. [S.l.]: Princeton University Press, 2007.

BENITEZ, J. "*A importância dos prazos de entrega no seu e-commerce*". 2021. Disponível em: <https://www.ecommercebrasil.com.br/artigos/prazos-de-entrega-no-e-commerce-importancia/>. Acesso em: 06 de junho 2022.

CAVALCANTE, H. da S. et al. Uma breve análise sobre a evolução da logística. s.d.

CAVICCHIOLLI, R. "*Os cinco maiores desafios para um e-commerce de sucesso*". 2021. Disponível em: <https://www.ecommercebrasil.com.br/artigos/os-cinco-maiores-desafios-para-um-e-commerce-de-sucesso/>. Acesso em: 06 de junho 2022.

CONHEÇA. "*Conheça os 4 principais tipos de logística*". 2022. Disponível em: <https://syscontrol.com.br/principais-tipos-de-logistica-existentes-no-mercado/>. Acesso em: 30 de maio 2022.

FORTNOW, L. The status of the p versus np problem. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 52, n. 9, p. 78–86, sep 2009. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/1562164.1562186>.

FREITAS, A. R. R. de. *Música evolutiva: uma abordagem computacional para composiçao algorıtmica*. Tese (Doutorado) — Universidade Federal de Ouro Preto, 2011.

GOLDBERG, D. E. Genetic algorithms in search optimization and machine learning. In: . [S.l.: s.n.], 1988.

HILLIER, F. S.; LIEBERMAN, G. J. *Introdução à pesquisa operacional*. [S.l.]: McGraw Hill Brasil, 2013.

HOFFMAN, K. L. et al. Traveling salesman problem. *Encyclopedia of operations research and management science*, Springer New York, v. 1, p. 1573–1578, 2013.

Juan Pablo D'Antiochia. "*Expansão dos métodos de pagamentos digitais no e-commerce da América Latina*". 2022. Disponível em: <https://www.ecommercebrasil.com.br/artigos/metodos-de-pagamentos-digitais/>. Acesso em: 06 de junho 2022.

KOSTER, R. D.; DUC, T. L.; ROODBERGEN, K. J. Design and control of warehouse order picking: A literature review. *European Journal of Operational Research*, p. 481–501, 10 2007.

LACERDA, E. G. de; CARVALHO, A. D. Introdução aos algoritmos genéticos. *Sistemas inteligentes: aplicações a recursos hídricos e ciências ambientais*, v. 1, p. 99–148, 1999.

LU, W. et al. An algorithm for dynamic order-picking in warehouse operations. *European Journal of Operational Research*, p. –, 07 2015.

MAHAJAN, R.; KAUR, G. Neural networks using genetic algorithms. *International Journal of computer applications*, Citeseer, v. 77, n. 14, 2013.

MILLER, C. E.; TUCKER, A. W.; ZEMLIN, R. A. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, ACM New York, NY, USA, v. 7, n. 4, p. 326–329, 1960.

NETTO, P. O. B. *Grafos: teoria, modelos, algoritmos*. [S.l.]: Editora Blucher, 2003.

NIENKOTTER, M. A. N. et al. Proposta de um modelo heurístico para a roteirização de empilhadeiras em um armazém de grande porte. Joinville, SC, 2017.

PACHECO, M. A. C. et al. Algoritmos genéticos: princípios e aplicações. *ICA: Laboratório de Inteligência Computacional Aplicada. Departamento de Engenharia Elétrica. Pontifícia Universidade Católica do Rio de Janeiro. Fonte desconhecida*, v. 28, 1999.

ROODBERGEN, K. J.; KOSTER, R. D. Routing order pickers in a warehouse with a middle aisle. *European Journal of Operational Research*, v. 133, p. 32–43, 08 2001.

SIPSER, M. *Introdução á teoria da computação*. [S.l.]: Cengage Learning, 2005.

TIPOS. "*Tipos de separação de pedidos*". 2017. Disponível em: <https://www.almmati.com/post/185/Tipos+de+separacao+de+pedidos>. Acesso em: 01 de junho 2022.